

DOJOBID — Operators Manual

Purpose: Day-to-day operation, administration, and troubleshooting **Audience:** DevOps, platform operators, support staff, and technical admins **Product:** DOJOBID marketplace (homeowners ↔ contractors) **Document date:** June 7, 2026

1. Introduction

This manual describes how to **run, monitor, configure, and support** the DOJOBID platform. It assumes the system is already built and deployed (or running locally for development). For architecture and schema details, see [DOJOBID-App-Build-Specification.pdf](#).

1.1 Platform Components

ComponentDefault URL (dev)Role __TABLE_SEP__ API [http://localhost:4000/api/v1](#) Backend services, auth, data Web [http://localhost:3000](#) Browser client + admin console MobileExpo Go / EAS buildsiOS and Android native app DatabaseNeon PostgreSQLPersistent data store

1.2 User Roles

RoleCapabilities __TABLE_SEP__ **HOMEOWNER**Post jobs, review bids, accept bids, message contractors **CONTRACTOR**Set service profile, search nearby jobs, submit bids, message after acceptance **ADMIN**View all users/jobs, ban/unban accounts via web admin or API

2. First-Time Setup

2.1 Prerequisites

1. **Node.js** ≥ 20
1. **npm** (ships with Node)
1. **PostgreSQL** connection string (Neon cloud or local)
1. **Expo Go** on iPhone/Android (for mobile dev testing)
1. Optional: AWS S3 credentials, Stripe keys (production)

2.2 Install Dependencies

From the repository root: `` `bash npm install cd apps/mobile && npm install && cd ../../` ``

2.3 Configure Environment

API – copy `apps/api/.env.example` to `apps/api/.env` : VariableRequiredDescription __TABLE_SEP__
DATABASE_URLYesPostgreSQL connection string JWT_ACCESS_SECRETYesAccess token signing secret
JWT_REFRESH_SECRETYesRefresh token signing secret API_PORTNoDefault 4000
API_CORS_ORIGINNoComma-separated allowed origins PAYMENTS_ENABLEDNo true to enable Stripe flows
JOBS_MAX_PHOTOSNoMax photos per job (default 4) STRIPE_SECRET_KEYIf paymentsStripe API secret
STRIPE_WEBHOOK_SECRETIf paymentsWebhook signature secret MEDIA_S3_BUCKETProd mediaS3 bucket name;
omit for dev-media mode **Web** – copy `apps/web/.env.local.example` to `apps/web/.env.local` : `
NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1` **Mobile** – copy `apps/mobile/.env.example` to
`apps/mobile/.env` : ` EXPO\_PUBLIC\_API\_URL=http://:4000/api/v1` Use your PC's Wi-Fi IP
(`ipconfig` on Windows). Physical phones cannot reach `localhost`.

2.4 Initialize Database

`bash npm run db:generate npm run db:migrate npm run db:seed` The seed creates three demo
accounts (password **Password123!**): EmailRole __TABLE_SEP__ homeowner@example.comHOMEOWNER
contractor@example.comCONTRACTOR admin@example.comADMIN

3. Starting and Stopping Services

3.1 Development (Three Terminals)

Terminal 1 – API ``bash npm run api`` Confirm: open `http://localhost:4000/api/v1/health` – expect `{"status":"ok",...}` . **Terminal 2 – Web** ``bash npm run web`` Open `http://localhost:3000` . **Terminal 3 – Mobile (Expo Go)** ``bash npm run mobile`` Scan the QR code in **Expo Go**. Use LAN mode (`--go --lan`); tunnel mode requires extra setup.

3.2 Production API

``bash cd apps/api npm run build npm run start:prod`` Run database migrations before starting:
``bash npm run prisma:deploy --workspace apps/api``

3.3 Stopping Services

Press **Ctrl+C** in each terminal. The API flushes in-flight requests on graceful shutdown. No separate stop script is required.

3.4 Startup Order

1. Database (Neon is always on; verify connectivity)

1. API

1. Web and/or Mobile clients

Clients will fail API calls if the backend is not running.

4. Health Checks and Monitoring

4.1 Health Endpoint

``GET /api/v1/health`` **Healthy response:** `json { "status": "ok", "time": "2026-06-07T12:00:00.000Z" }` Use this endpoint for load balancer probes and uptime monitors. It does not verify database connectivity – add a separate DB check in production if needed.

4.2 Feature Flags Endpoint

``GET /api/v1/flags`` Returns public runtime flags (no auth required): `FlagMeaning __TABLE_SEP__ paymentsEnabledStripe acceptance fees active messagingGroupVisibleGroup messaging UI enabled jobsMaxPhotosMaximum photos allowed per job` Change flags via API environment variables and restart the API.

4.3 Logs

The API logs to stdout. In development, NestJS prints request errors and bootstrap messages. In

production, pipe stdout to your log aggregator (CloudWatch, Datadog, etc.).

4.4 Database Inspection

Open Prisma Studio for a GUI view of records: ``bash npm run prisma:studio --workspace apps/api``
Default URL: `http://localhost:5555` . **Do not expose Studio to the public internet.**

5. Feature Flags and Configuration

Feature flags are read from environment variables at API startup. Env VariableDefaultEffect
__TABLE_SEP__ PAYMENTS_ENABLEDfalseEnables Stripe payment sessions for bid acceptance fees
MESSAGING_GROUP_VISIBLEfalseShows group chat option to all bidders
PROFILE_REQUIRE_VERIFICATIONfalseBlocks unverified users from certain actions
JOBS_MAX_PHOTOS4Limits photo uploads per job **To change a flag:**

1. Edit `apps/api/.env` (or production secrets manager)

1. Restart the API process

1. Verify via `GET /api/v1/flags`

Flags require an API restart today. For production, consider moving flags to a database table or feature-flag service for live toggling without redeploy.

6. Operator Workflows by Role

6.1 Homeowner Workflow

1. Register or log in (web or mobile)
1. **Post a Job** – enter title, trade type, description, address, budget, photos
1. Wait for contractor bids (notifications appear in Activity)
1. Review bids on job detail screen
1. **Accept a bid** – full address revealed to both parties; optional payment if enabled
1. Message the contractor via the job thread
1. Close or cancel the job when finished

Web paths: `/jobs/new` , `/my-jobs` , `/jobs/[id]` **Mobile tabs:** My Jobs → Job Detail

6.2 Contractor Workflow

1. Register as CONTRACTOR and complete **Profile** (trades, service radius, GPS location)
1. Open **Find Jobs** tab (not visible to homeowners)

1. Set search radius (10 / 25 / 50 miles) and trade filters
1. Pull to refresh or tap search; jobs show ZIP code only until bid accepted
1. Submit bid with amount and message
1. If accepted, view full address and message homeowner

Web paths: /jobs (search), /jobs/[id] **Mobile tabs:** Find → Job Detail

6.3 Admin Workflow

1. Log in as admin (admin@example.com in dev)

1. Open **Admin console** at <http://localhost:3000/admin>

1. **Jobs tab** – view all platform jobs
1. **Users tab** – view users; ban or unban accounts
1. Banned users cannot authenticate; existing sessions are rejected on next request

API equivalents (require ADMIN JWT): ActionMethodPath __TABLE_SEP__ List jobsGET /admin/jobs
List usersGET /admin/users Ban userPOST /admin/flags/ban-user Unban userPOST /admin/flags/unban-user
Ban/unban actions are recorded in the **audit_logs** table.

7. Database Operations

7.1 Common Commands

TaskCommand __TABLE_SEP__ Regenerate Prisma client npm run db:generate Apply dev migrations npm run db:migrate Deploy migrations (prod) npm run prisma:deploy --workspace apps/api Seed demo data npm run db:seed Open Prisma Studio npm run prisma:studio --workspace apps/api

7.2 Re-seeding Demo Data

The seed script upserts demo users and recreates sample jobs. Safe to re-run: ``bash npm run db:seed`` Existing demo jobs for seed users are deleted before recreation.

7.3 Backups (Production)

Neon provides automated backups on paid tiers. For manual backup: ``bash pg_dump "$DATABASE_URL" > dojobid-backup-$(date +%Y%m%d).sql`` Store backups encrypted and test restore procedures quarterly.

7.4 Schema Changes

1. Edit apps/api/prisma/schema.prisma

1. Run npm run db:migrate locally and test

1. Deploy with prisma migrate deploy in CI/CD before API rollout

8. Media and File Storage

8.1 Development Mode (No S3)

When MEDIA_S3_BUCKET is empty, the API uses local **dev-media** storage:

1. Uploads go to .dev-media/ in the API working directory

1. Served at `/api/v1/dev-media/`

1. Signed URLs use the request Host header (supports LAN IP for mobile)

Operator notes:

1. `.dev-media/` is gitignored; contents are lost on clean checkout

1. Do not use dev-media in production
1. Max upload size: 12 MB per file

8.2 Production Mode (S3)

Set AWS credentials and bucket env vars. Flow:

1. Client calls `POST /media/sign-upload`

1. Client PUTs file to presigned S3 URL

1. Job stores the returned `fileUrl`

Ensure bucket CORS allows client origins and lifecycle rules manage old uploads if needed.

9. Payments (Stripe)

Payments are disabled by default (`PAYMENTS_ENABLED=false`).

9.1 Enabling Payments

1. Set `PAYMENTS_ENABLED=true`

1. Set STRIPE_SECRET_KEY and STRIPE_WEBHOOK_SECRET

1. Configure Stripe webhook endpoint: POST https:///api/v1/payments/webhook/stripe

1. Restart API

9.2 Acceptance Fee Model

When enabled, bid acceptance may require:

1. HOMEOWNER_ACCEPT_FEE – homeowner pays on accept
1. CONTRACTOR_ACCEPT_FEE – contractor pays on accept

Payment status is tracked in the payments table. Failed payments leave the bid in a pending state until resolved.

10. Mobile App Operations

10.1 Expo Go (Development)

StepAction __TABLE_SEP__ 1Ensure API is running and reachable from phone 2Set EXPO_PUBLIC_API_URL to LAN IP (not localhost) 3Run npm run mobile from repo root 4Open Expo Go, scan QR code 5Reload app after any .env change **Common issue:** Phone shows network errors → verify LAN IP with ipconfig, confirm PC firewall allows port 4000.

10.2 EAS Builds (Staging / Production)

Configuration: apps/mobile/eas.json ProfileUse case __TABLE_SEP__ developmentDev client on physical device development-simulatoriOS Simulator dev client previewInternal distribution productionApp Store / Play Store `bash cd apps/mobile npm run eas:login npm run eas:build:ios:dev # iOS dev build npm run eas:build:android:dev # Android dev build` Custom app icons and native modules require an EAS build – Expo Go uses the default Expo shell.

10.3 Push Notifications

Device tokens are stored via POST /devices . Full push delivery (FCM/APNs) is not yet wired; tokens are collected for future use.

11. Web Application Operations

11.1 Development

`bash npm run web` Runs Next.js on port 3000 with hot reload.

11.2 Production Build

`bash npm run build --workspace apps/web npm run start --workspace apps/web` Set NEXT_PUBLIC_API_URL to the production API URL before building – Next.js embeds public env vars at build time.

11.3 Admin Console

URL: /admin Requires login as ADMIN role. Provides job listing, user listing, and ban/unban controls.

12. Realtime (Socket.IO)

The API exposes a Socket.IO gateway at /realtime . Clients authenticate with JWT on connect.

Operator checklist:



1. Ensure WebSocket upgrades are allowed through your reverse proxy (nginx: proxy_http_version 1.1 , Upgrade headers)

1. CORS origins must include web client URLs

1. If realtime is unavailable, REST polling still works for messages and notifications (degraded UX)
-

13. Troubleshooting Guide

13.1 API Won't Start

SymptomLikely causeFix __TABLE_SEP__ Prisma client errorClient not generated npm run db:generate
Database connection refusedWrong DATABASE_URLVerify Neon dashboard, IP allowlist Port in
useAnother process on 4000Change API_PORT or kill conflicting process

13.2 Mobile Can't Reach API

SymptomFix __TABLE_SEP__ Network request failedUse LAN IP in EXPO_PUBLIC_API_URL 401 on all
requestsLog out and log back in; tokens may be stale Photos don't displayConfirm dev-media is
running; check API logs for PUT errors

13.3 Login Failures

SymptomFix __TABLE_SEP__ Invalid credentialsVerify email/password; re-seed if needed Account
bannedAdmin must unban via /admin or API CORS error in browserAdd web origin to
API_CORS_ORIGIN

13.4 Job Search Returns No Results

1. Contractor profile must have base_lat/lng set (Profile screen → Use GPS)

1. Increase search radius (10 / 25 / 50 miles)

1. Verify jobs exist with matching work_type and OPEN status

1. Seed data jobs are in NYC metro area – contractor base location must be nearby

13.5 Stripe Webhook Failures

1. Confirm raw body is preserved (configured in main.ts)

1. Verify webhook secret matches Stripe dashboard
1. Check API logs for signature verification errors

13.6 Regenerate PDF Documentation

```
From docs/ : `bash node generate-pdf.mjs operators-manual node generate-pdf.mjs build-spec`
```

14. Production Deployment Checklist

1. ☐ Set strong `JWT_ACCESS_SECRET` and `JWT_REFRESH_SECRET` (unique, random, ≥ 32 chars)

1. ☐ Configure production `DATABASE_URL` on Neon with connection pooling

1. ☐ Run `prisma migrate deploy` before API start

1. ☐ Set `API_CORS_ORIGIN` to production web URL(s) only

1. ☐ Configure S3 bucket and IAM credentials for media

1. ☐ Set `PAYMENTS_ENABLED` and Stripe keys if accepting fees

1. ☐ Enable HTTPS on API and web (TLS termination at load balancer)

1. [] Configure health check on `/api/v1/health`

1. [] Set up log aggregation and error alerting

1. [] Verify Neon backup retention policy

1. [] Build mobile with EAS production profile; submit to app stores

1. [] Rotate demo/seed passwords – do not ship `Password123!` to production

15. Security Operations

PracticeDetail __TABLE_SEP__ Password policyArgon2id hashing; enforce strong passwords in registration UI Token rotationRefresh tokens rotate on use; logout invalidates refresh token Ban enforcement is_banned checked on every authenticated request Location privacyFull addresses hidden until bid acceptance – verify serializers after schema changes SecretsNever commit `.env` files; use secrets manager in production Admin accessLimit ADMIN accounts; audit ban actions in audit_logs

16. Support Reference

Quick Links (Development)

ResourceLocation __TABLE_SEP__ API health `http://localhost:4000/api/v1/health` Feature flags `http://localhost:4000/api/v1/flags` Web app `http://localhost:3000` Admin console `http://localhost:3000/admin` Prisma Studio `http://localhost:5555` (when running)

Demo Credentials (Development Only)

Password for all: `Password123!` AccountEmail __TABLE_SEP__ Homeownerhomeowner@example.com Contractorcontractor@example.com Adminadmin@example.com

Work Types (Search Filters)

electrical · plumbing · landscaping · hauling · carpentry · handyman · other

17. Document Revision History

DateChange __TABLE_SEP__ 2026-06-07Initial operators manual

For architecture, data model, and API reference, see [DOJOBID-App-Build-Specification.pdf](#).